
JobTracker Hub

User Guide & Workflow Playbook

What it is. JobTracker Hub is a local-first dashboard for the job-application documents you already keep on disk. It builds a searchable application index, gives you a real pipeline and triage workflow, and keeps the underlying files under your control.

Runs locally	FastAPI web UI; no account required.
Data model	Your existing folders + two local SQLite databases.
Privacy posture	Binds to localhost; no cloud service is required by the application.
License	MIT (see repository LICENSE).

Repository: JobTracker Hub — this guide lives in **docs/** alongside the code it documents.

Guide version: 1.1.0 | **Updated:** August 28, 2026

Designed as a JobTracker Hub reference manual, with a compact technical-document layout.

Contents

1 Start Here	2
2 How Your Folder Becomes a Tracker	5
3 First Build: What to Expect	5
4 The Web Dashboard	6
5 Working With Documents	9
6 Command Palette and Keyboard Workflow	10
7 Multiple Trackers and Workspaces	10
8 Privacy and Local Data	15
9 A Recommended Daily Workflow	16
10 Troubleshooting	16
11 License	18
A Repository Map for Contributors	18
B Repository Safety Notes	18
C Release Notes for 1.1.0	18

Quick Navigation — jump to the task you need. The PDF table of contents and bookmarks are clickable. If you are new to JobTracker Hub, start with installation or the quick start, then use the task links below as your day-to-day map.

Install the Mac app	Installing the macOS desktop app
Start with your existing folder	Creating and importing trackers
Understand the dashboard	The Web Dashboard
Check an import before committing	Import preview and validation
Understand and back up your data	Database management and local storage
Build a simple daily routine	A Recommended Daily Workflow
Something went wrong	Troubleshooting
Contributing or inspecting the repository	Appendix A — Repository Map for Contributors

1 Start Here

1.1 Release overview

JobTracker Hub 1.1.0. This release refines the desktop and web workflows around workspace health, import validation, and durable local data. It also documents the macOS DMG installation path and the safeguards used when importing folders or archives.

The 1.1.0 guide covers the current status cards, the multi-step import preview, persistent `overrides.db` tracking data, and the packaged macOS installation flow. The underlying design remains local-first: the application works with documents on your filesystem and keeps tracking state in local storage.

1.2 What JobTracker Hub actually does

JobTracker Hub sits one level inside a folder that already contains your job-search material. It indexes the files on disk, infers useful metadata from folder and filename conventions, and presents the result through a local dashboard.

The important design choice is that the application does *not* require you to migrate your existing documents into a hosted database. Your tracker remains a normal folder you can inspect in Finder or Explorer.

The core mental model: your filesystem is the source of truth. `jobtracker.db` is a rebuildable index, while `overrides.db` holds the durable choices you make in the app, such as status overrides, notes, dates, snoozes, archives, and company merges.

1.3 Quick start

The repository includes a fake `sample-tracker/` so you can verify the installation before pointing the application at real documents.

1. Copy `sample-tracker/` to a new folder, for example `my-tracker/`.
2. Copy the repository's `_app/` directory into `my-tracker/`.
3. Create a virtual environment and install the dependencies.
4. Start the recommended web application.
5. Open <http://127.0.0.1:8000> and choose **Build index** the first time.

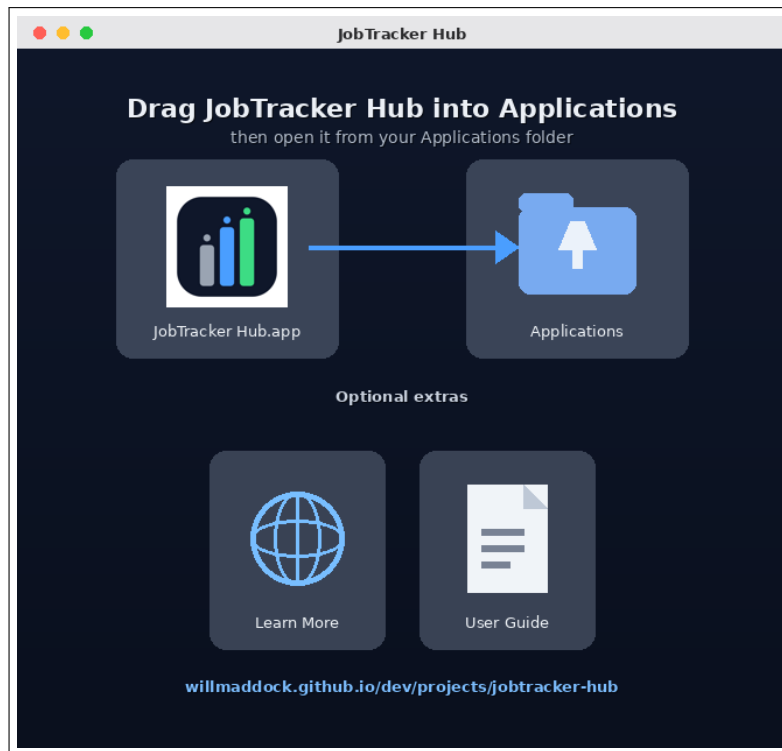
```
cp -r sample-tracker my-tracker
cp -r _app my-tracker/_app
cd my-tracker/_app
python3 -m venv .venv
source .venv/bin/activate      # Windows: .venv\\Scripts\\activate
pip install -r requirements.txt
uvicorn api:app --reload --port 8000
```

The frontend's requests are relative to that origin, so open the app at <http://127.0.0.1:8000> rather than opening `frontend/index.html` directly as a file — a bare file won't be able to reach the API.

1.4 Installing the macOS desktop app

If you were given a `JobTracker Hub.dmg` instead of the repository, you don't need Python, a virtual environment, or a terminal at all — the desktop build packages the backend and frontend together into a single double-clickable app.

1. Double-click **JobTracker Hub.dmg** to mount it. A window opens showing the **JobTracker Hub** icon and an **Applications** shortcut, connected by an arrow.
2. Drag the **JobTracker Hub** icon onto the **Applications** shortcut to install it.
3. Eject the mounted disk image (the **JobTracker Hub** entry under **Locations** in Finder's sidebar), then open the app from your **Applications** folder as usual.

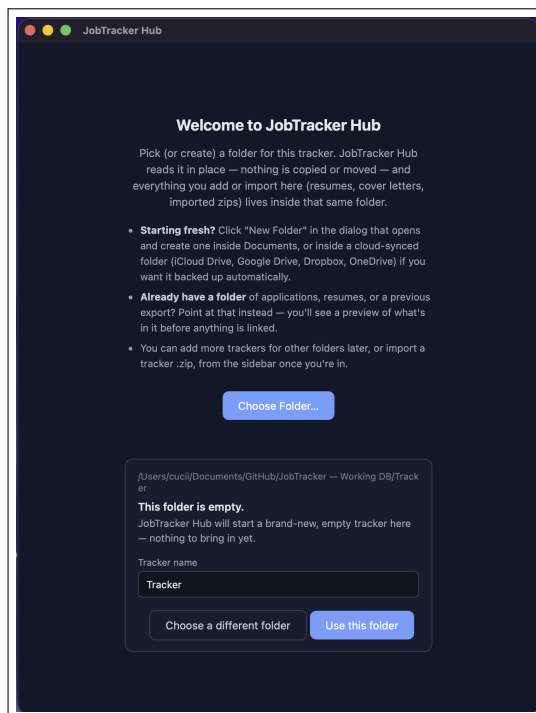


The installer window: drag the app icon onto the Applications shortcut to install. Two optional shortcuts sit below it – **Learn More** and **User Guide**.

Below the main drag-and-drop row, the same window has two optional shortcuts that don't affect the install itself:

- **Learn More** opens the project's portfolio page in your browser – it's an Internet shortcut, not part of the app.
- **User Guide** is this document. Double-click it to open the PDF directly from the install window, before the app is even copied to **Applications**.

First launch. The very first time you open the app, it shows a welcome screen and a **Choose Folder...** button — this becomes your tracker root, exactly like the **my-tracker/** folder in the command-line quick start above. After you pick a folder, the app previews what it found there (empty, an existing tracker, or something else entirely) before anything is committed; edit the tracker name if you like, then click **Use this folder** to confirm, or **Choose a different folder** to pick again. The app remembers this choice and opens straight to your dashboard on every future launch.



The first-run screen after picking a folder: JobTracker Hub previews what it found before you commit.

"Not Opened" warning on first launch. This is normal for an app downloaded outside the App Store and not notarized by Apple — it is not a sign anything is actually wrong. The very first time you open **JobTracker Hub**, macOS shows a dialog titled "**JobTracker Hub.app** Not Opened" with two buttons: **Move to Trash** and **Done**. There is no **Open Anyway** button in this first dialog, so:

1. Click **Done** — *not* **Move to Trash**.
2. In **Applications**, right-click (or Control-click) **JobTracker Hub** and choose **Open** from the menu. A second dialog appears; this one has an **Open** button — click it.
3. If right-clicking doesn't offer **Open**, go to **System Settings > Privacy & Security**, scroll to the **Security** section, and click **Open Anyway** next to the message about **JobTracker Hub** being blocked.

You only need to do this once — macOS remembers your choice for future launches.

Unlike the command-line install, the desktop app keeps its app-level writable state in a per-user Application Support folder rather than next to the code, since the installed `.app` bundle itself is read-only. Tracker-owned durable data remains with the tracker, as described in Section 8.2:

```
~/Library/Application Support/JobTracker Hub
```

See Section 10 if the app doesn't open, or opens to a blank window.

1.5 Requirements

- Python 3.10 or newer is recommended; use the same Python 3 version for the application and test environments.
- A modern web browser for the web UI.
- A local filesystem containing the documents you want to track.

1.6 Running the app

JobTracker runs as a single FastAPI process, `uvicorn api:app -port 8000`, which both exposes the backend as JSON and serves the frontend (`frontend/index.html`) itself. Open `http://127.0.0.1:8000` in your browser — the frontend’s requests are relative to that origin, so it needs to be loaded from the running server rather than opened as a bare file. This gets you the full feature set: the dark master-detail UI, Kanban, command palette, document operations, and workspaces.

2 How Your Folder Becomes a Tracker

2.1 The recommended layout

The application expects `_app/` to live directly inside the folder it should index:

```
MyTracker/
|-- Applications/
|   |-- Acme Robotics/
|   |   |-- resume.pdf
|   |   |-- coverletter.pdf
|   |-- Riverbend Public Library/
|   |   |-- interview-notes.txt
|-- Certifications/
|-- References/
|-- Resume Library/
'-- _app/
    |-- api.py
    |-- db.py
    |-- build_index.py
    |-- classify.py
    '-- frontend/index.html
```

The tracker root is determined automatically: the application uses the parent directory of `_app/`. There is normally no path you need to enter into a settings screen.

2.2 What the indexer ignores

The filesystem walker deliberately skips the application’s own `_app/` directory, hidden files and folders, common virtual environments, dependency folders, archives, and database files. This prevents the tracker from turning its own implementation or backups into job documents.

2.3 What counts as an application

Under `Applications/`, the standard grouping is based on the folder hierarchy. In the common case, the company folder is the company and the next role folder becomes the role label. That means you can organize documents naturally instead of entering every document into a separate form.

3 First Build: What to Expect

When you click **Build index**, JobTracker walks the tracker root and rebuilds the disposable index from the files currently on disk.

Rebuild is not a reset of your personal tracking data. `jobtracker.db` is regenerated. `overrides.db` is intentionally preserved so notes, manual status changes, dates, snoozes, archives, and company merges survive an index rebuild.

A first rebuild is the point where you should check whether your folder names and filenames are being classified the way you expect.

3.1 Classification is intentionally simple

JobTracker primarily uses filename and folder heuristics rather than opening every PDF and trying to understand its prose. This keeps indexing fast and predictable.

Typical filename signals include:

Filename signal	Likely document type
resume	Resume
cover letter, coverletter	Cover letter
interview, phone screen, screener, schedule	Interview / scheduling material
cheat sheet, interview prep, mock, study	Interview preparation
reject, not referred	Rejection / non-referral
thank you, application received, confirmation	Application confirmation
job bulletin, job description, job id	Job posting

3.2 Customize classification with one file

The primary configuration file is `_app/classify_config.json`. It contains section rules and an optional nested-application pattern for compliance folders.

This is the file to customize first. The generic classifier in `classify.py` is intended to work out of the box and generally should not need editing unless your filename conventions are unusual.

After changing classification rules, rebuild the index. Your `overrides.db` is not rewritten by that rebuild.

4 The Web Dashboard

The web UI is the recommended interface because it exposes the full workflow in one place.

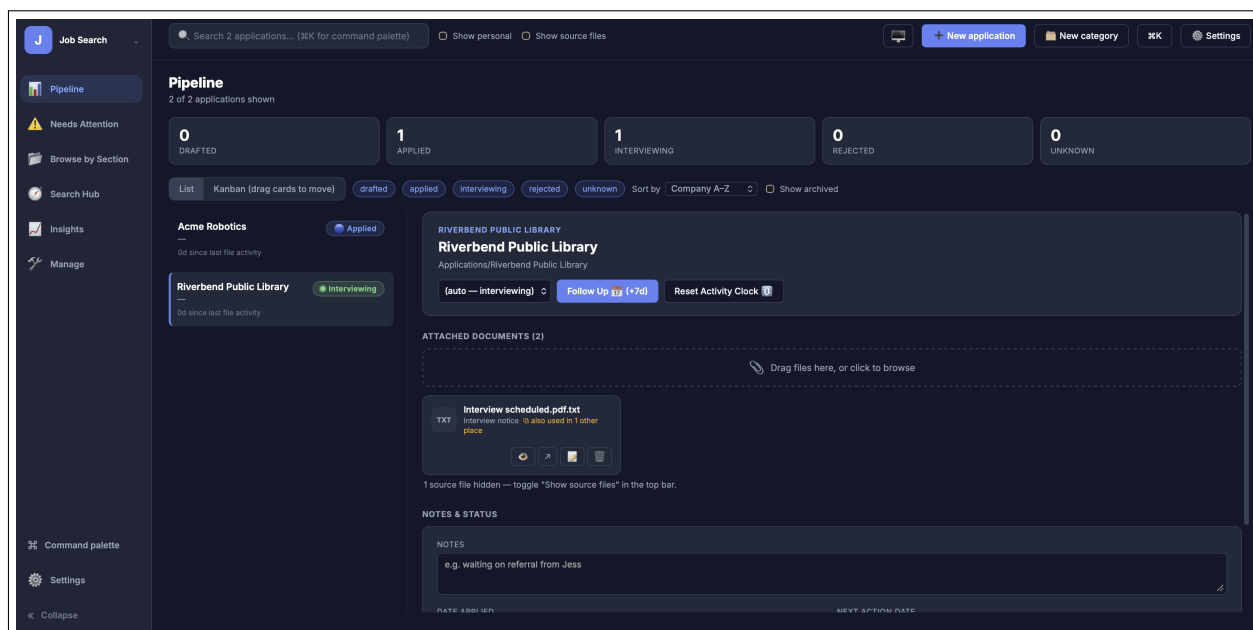
4.1 Pipeline

The **Pipeline** is the main application-management view. It supports both a list representation and a drag-and-drop Kanban board grouped by status.

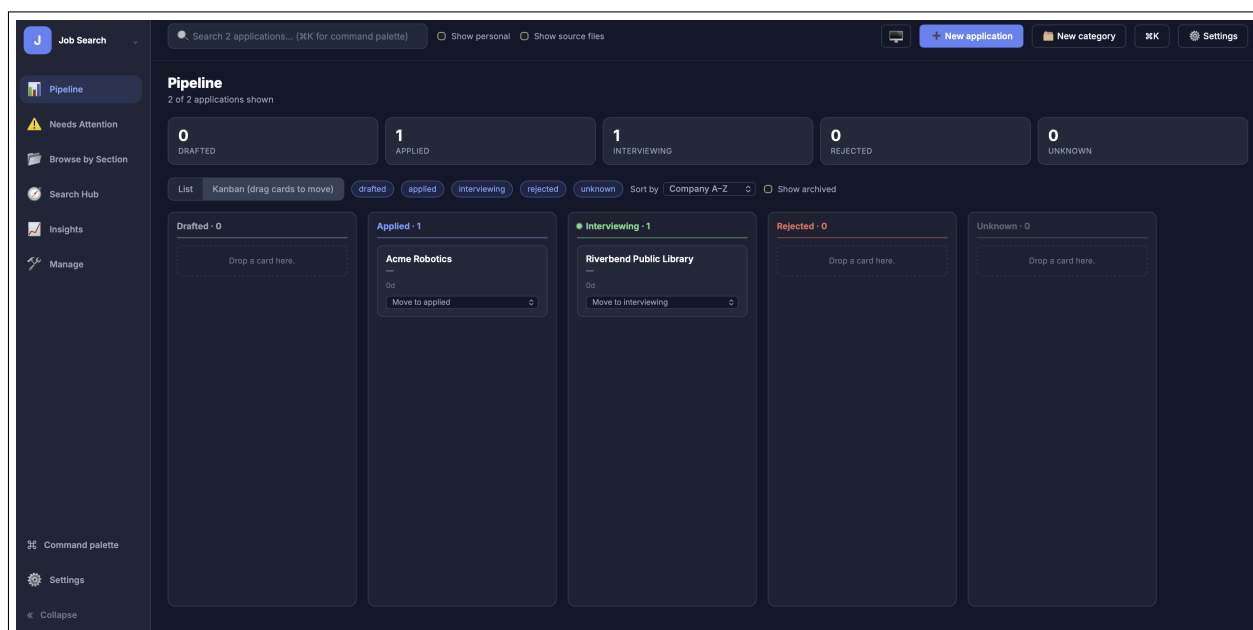
A typical application card can surface:

- company and role information,
- how long the application has been sitting,
- staleness / follow-up indicators,
- attached documents,
- status and manual overrides,
- application dates, notes, and next actions.

Dropping a card into a different Kanban column updates the status immediately.



The Pipeline in list view: status counts, filters, and the detail pane for the selected application.



The same Pipeline in Kanban view — drag a card between columns, or use each card's status dropdown.

4.2 Workspace status cards

The dashboard status cards provide a real-time summary of the active workspace. They are intended to answer the quick health questions you would otherwise have to reconstruct from several views: how much data is indexed, what needs attention, and whether the workspace is ready for normal use.

The cards update as the current workspace changes and should be read as operational indicators rather than permanent historical records. After adding, moving, or importing documents, rebuild the index when prompted or when the dashboard needs to reflect filesystem changes.

- **Volume and index metrics** summarize the records currently known to the workspace.
- **Pipeline metrics** show the current distribution of tracked applications and their statuses.

- **Attention metrics** surface stale or due work that may require follow-up.
- **Workspace health** helps distinguish a healthy, populated workspace from one that still needs an initial build, import confirmation, or other setup step.

Use the cards as a first check. If a count looks unexpectedly low after a filesystem change, do not assume data was deleted. Check the workspace and rebuild the disposable index before changing durable tracking state.

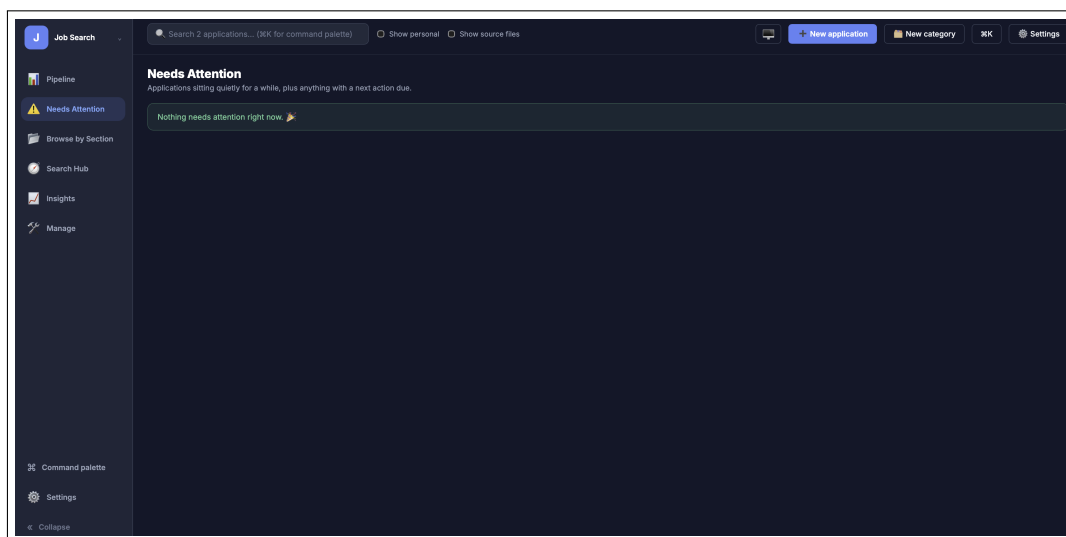
4.3 Needs Attention

Needs Attention is the triage queue. It identifies applications that have gone quiet and applications whose next-action date has arrived.

The default staleness thresholds in the current implementation are:

- applied or interviewing: 21+ days without an update,
- drafted or unknown: 14+ days without an update.

From this view you can mark follow-up work, snooze an item for seven days, archive it, or perform supported bulk actions. Archive is intentionally non-destructive: it changes tracking state rather than deleting your files.



Needs Attention with an empty queue — the state you want to see most mornings.

4.4 Search

Search covers indexed filename, company, role, and full-text search data exposed by the backend. Matching terms are highlighted in the results as you type.

This is particularly useful when the folder tree is large and you remember a company, title, keyword, or document name but not its exact location.

4.5 Browse

Everything outside **Applications/** can be surfaced as configurable sections, such as credentials, network, resume library, leads, compliance, personal material, or your own categories.

The classifier creates a stable section identifier from custom top-level folder names when no explicit mapping exists.

4.6 Search Hub

Search Hub is a curated directory of external job-search sites — higher-ed, K-12, government, mainstream job boards, and AI-powered job-search assistants — grouped by category, so you're not re-Googleing the same list every week.

Set your **target role** and **location** once at the top of the page and Search Hub uses them to build a ready-to-paste AI prompt plus per-site search links across every card below.

- Cards with a built-in link go straight to **Open** or **Search**; cards without one prompt you to save your own via the pencil icon.
- Add your own custom cards, or override any built-in card's title or URL, with **Reset all to defaults** available if you want to start over.

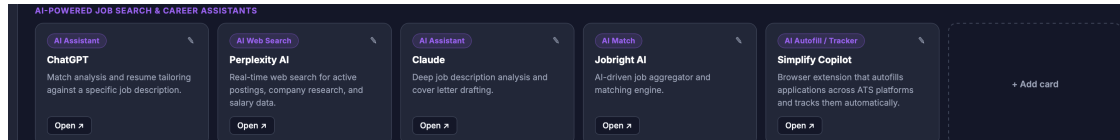


Figure 1: Search Hub with the AI-powered job-search and career-assistant resources.

Everything here — your target role/location, custom cards, and overrides — is saved server-side with the rest of your tracker's data, so it travels with the tracker through export and import instead of being pinned to one browser.

4.7 Insights

Insights turns the indexed application history into a small set of useful metrics, including:

- response rate,
- interview rate,
- average time to response,
- application velocity over time,
- status distribution.

Tip: set a real **Date applied** on active applications. File modification time can change when documents are copied, synced, or reorganized, while Date applied preserves the meaning of the application timeline and improves the velocity analysis.

4.8 Manage

Manage handles higher-level housekeeping:

- merge companies that were split across multiple folder names,
- undo a merge,
- review archived applications,
- restore archived items.

Company merges affect the application's logical view; they do not require you to rename the underlying files.

5 Working With Documents

5.1 Upload

The web app allows documents to be dragged onto an application. New files are written into the application's folder and become visible after the index is updated.

5.2 Rename

Documents can be renamed in place through the web interface. This changes the real file on disk, not merely a label in the database.

5.3 Delete safely

Delete operations are designed around the operating system Trash rather than a permanent unlink. The current implementation also exposes an undo affordance before the trash move is committed.

Still treat delete as a real filesystem operation. The app's safety behavior is better than a hard delete, but the file you are working with is your actual document. Verify the target before confirming a destructive action.

5.4 Source files

.tex source files are indexed and searchable, but they are hidden from normal document lists by default. Enable **Show source files** when you want to work with LaTeX sources alongside their generated PDFs.

5.5 Inline previews

The web app can open supported PDFs in a side drawer and can preview DOCX content through the backend's document-preview path. Text-based sources are also surfaced where appropriate.

6 Command Palette and Keyboard Workflow

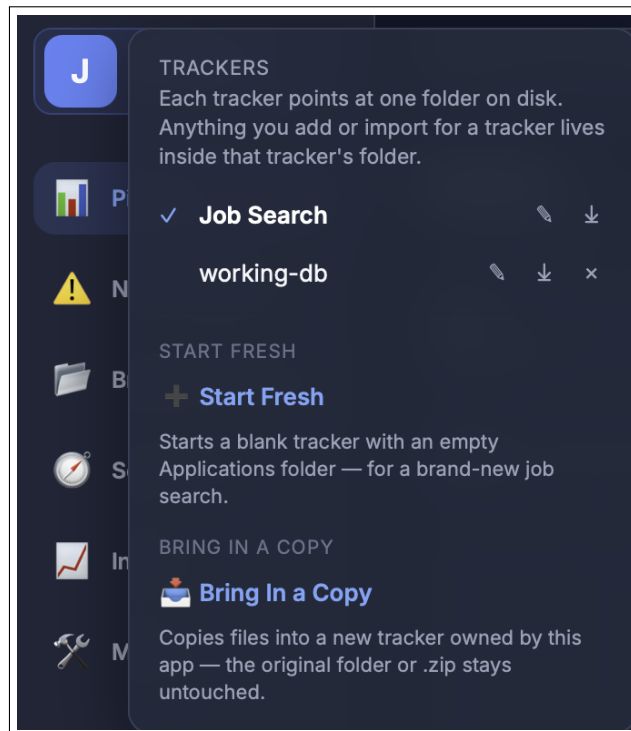
The web UI includes a command palette:

Shortcut	Action
Cmd+K / Ctrl+K	Open the command palette
j / k	Move focus in list views
Arrow keys	Move through list items
Enter	Open the first PDF inline
Esc	Close the active drawer / view

The command palette also exposes view jumps, application search, and the rebuild action.

7 Multiple Trackers and Workspaces

A single running JobTracker instance can manage more than one tracker through its workspace switcher — click the **J** logo, top-left, to open it. Every tracker you have is listed there, with a checkmark on the active one; clicking a different row switches to it instantly. Switching does not start a second server or duplicate the running application — it repoints the one running instance at a different data folder and database pair.



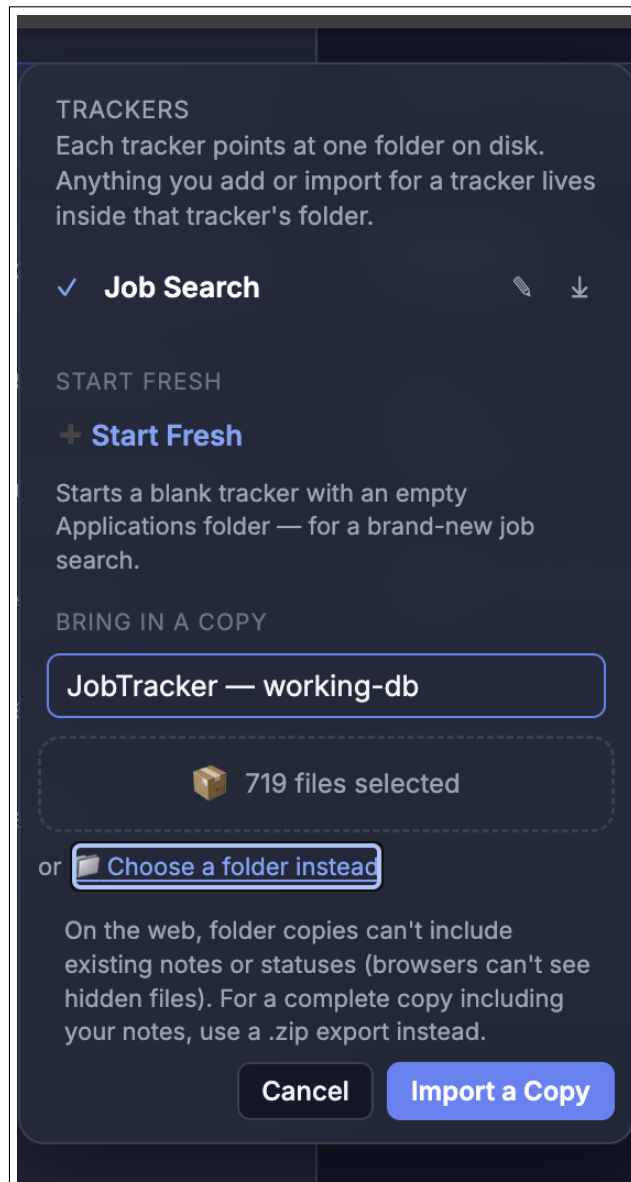
The tracker switcher. Each row has rename (pencil) and export (download) actions; every tracker but the first can also be deleted.

7.1 Creating and importing trackers

The switcher offers up to three ways to add a tracker, grouped by intent rather than by mechanism, depending on how you're running JobTracker Hub:

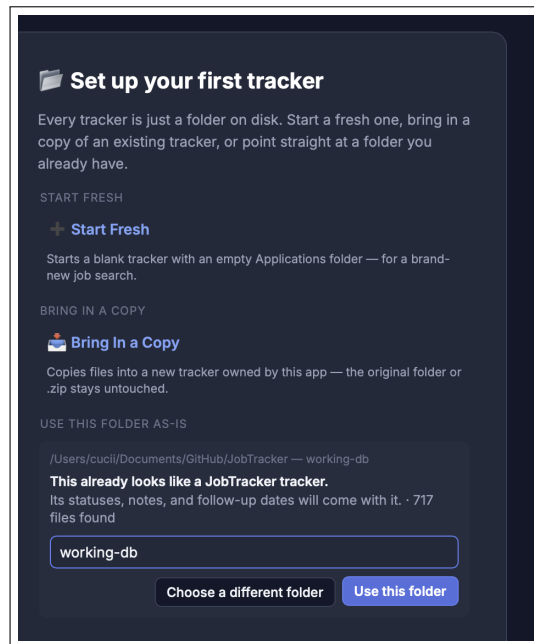
- **Start Fresh** starts a blank tracker with an empty Applications folder, as a sibling folder next to your original one — for a brand-new job search.
- **Bring In a Copy** brings an existing folder of documents or a **.zip** export into a new tracker owned by this app — the original folder or **.zip** stays untouched. Use the dropzone for a **.zip**, or choose **Choose a folder instead** for a folder. In the packaged desktop app, the folder path uses the same preview-before-commit safety step described in Section 7.3.
- **Use This Folder As-Is** (desktop app only) points a new tracker straight at a folder in place — nothing is copied, and its notes and statuses (if any) come with it. It's the same pick-preview-confirm flow as first launch, just reachable again later from the switcher.

All three paths apply the same filtering — the application's own **_app/** directory is never carried into a new tracker, even if it happens to be present in what you're importing or linking.



The switcher's **Bring In a Copy** panel, with a tracker name entered and a folder already selected (719 files) via **Choose a folder instead**.

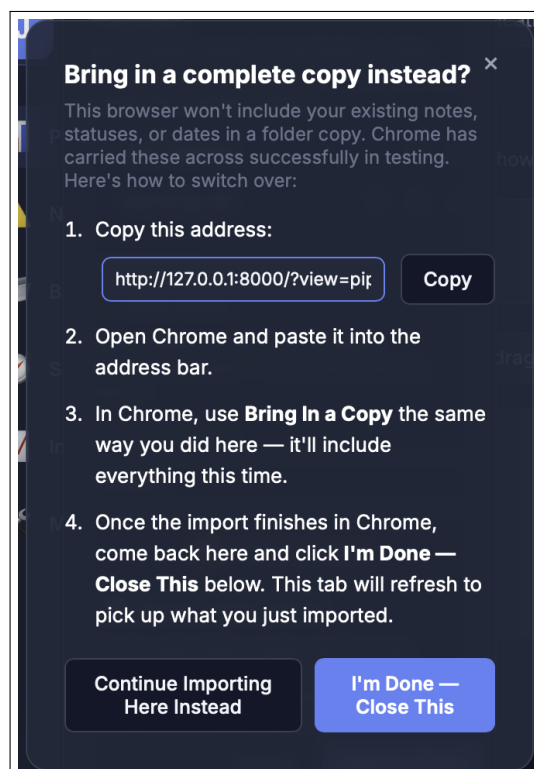
On the packaged desktop app, pointing **Use This Folder As-Is** at a folder that already looks like a JobTracker tracker is detected automatically — the panel calls this out explicitly, along with what it will carry over, before you commit:



The packaged app detecting an existing tracker: “This already looks like a JobTracker tracker. Its statuses, notes, and follow-up dates will come with it.”

7.2 Bringing a browser folder import to Chrome

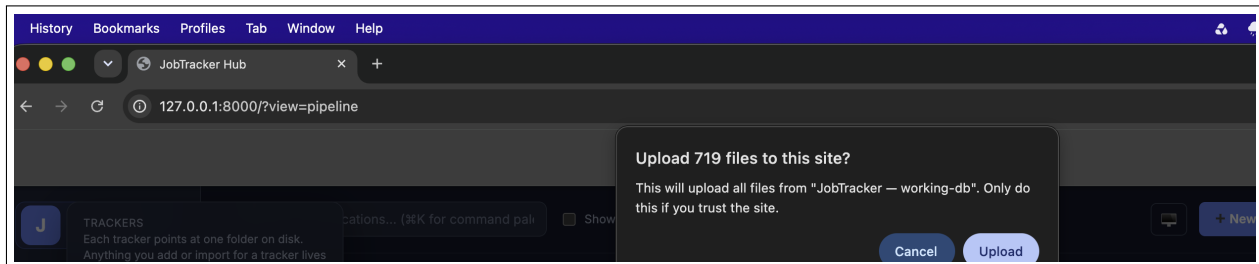
Browsers can’t see hidden files, so a plain folder copy made through **Bring In a Copy** in a non-Chrome browser can’t include your existing notes, statuses, or dates — only Chrome has been confirmed to carry those across successfully. To avoid that data loss, JobTracker Hub offers a handoff before the copy happens: if you pick a folder (not a .zip) for **Bring In a Copy** in a browser other than Chrome, and that folder isn’t already a fresh, tracker-less pick, a modal opens instead of proceeding with the import.



The Chrome handoff modal: copy the address, open Chrome, redo **Bring In a Copy** there, then close the modal to refresh this tab.

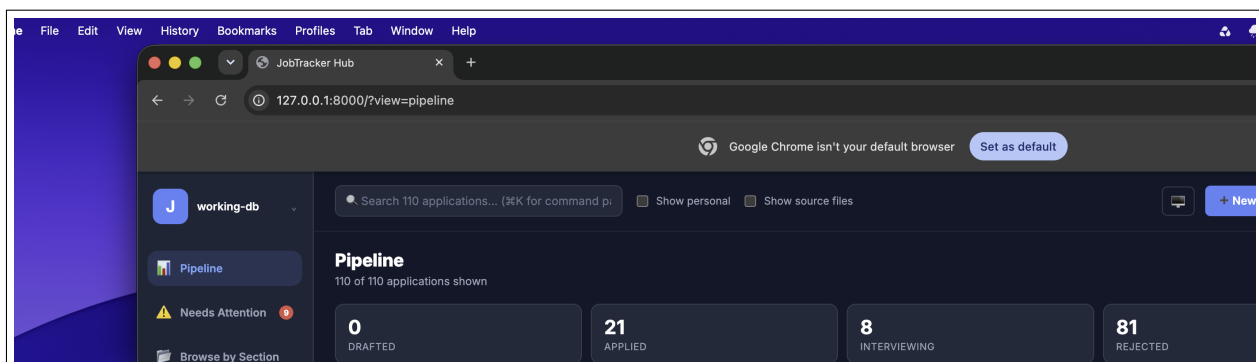
The modal walks through four steps: copy the destination address, open Chrome and paste it into the address bar, use **Bring In a Copy** again from inside Chrome (which will include everything this time), then come back and click **I'm Done — Close This** once the import finishes in Chrome — this tab reloads automatically to pick up what you just imported. If you'd rather not switch browsers, **Continue Importing Here Instead** proceeds with today's browser and the same folder-copy limitation as before.

Chrome itself will ask you to confirm the upload before it proceeds:



Chrome's own upload confirmation for a folder import — reflects however many files the picked folder actually contains.

Once the import finishes in Chrome, the new tracker is fully populated there, and switching back to your original browser and reloading will show the same data, since both are reading from the same tracker folder on disk:



The imported tracker in Chrome after a real round trip: full Pipeline counts and a confirmation toast that notes, statuses, and dates were carried over.

7.3 Import preview and validation

Folder and archive imports use a preview-first workflow. The preview is deliberately separate from the final import so that you can inspect what JobTracker found before records are written into the destination workspace.

1. **Choose the source.** Select a **.zip** export or choose a folder directly, depending on the import path available in your build.
2. **Review the preview.** JobTracker scans the source and presents the detected tracker contents before committing the import.
3. **Validate the records.** Check the proposed tracker name, source contents, application/document counts, and any warnings or exclusions shown by the preview.
4. **Confirm the import.** Click **Use this folder** for a folder-based import, or the corresponding import confirmation control for an archive. Nothing should be treated as accepted until you explicitly confirm.

5. **Open and verify.** After import, open the new workspace and confirm that the expected documents and application records are present. Rebuild the index if the source changed while you were reviewing it.

Preview before commit. The preview is the safety boundary for an import. If the source, counts, or proposed destination do not look right, choose a different source rather than confirming and trying to repair the workspace afterward.

7.4 Exporting and deleting a tracker

Export (the download-arrow icon on a tracker's row) zips up that tracker's data — **Applications/**, **Certifications/**, **References/**, and so on — and downloads it, always excluding **_app/** itself. This is the supported way to back up a tracker or hand its data to a genuinely separate installation.

Delete removes a tracker from the switcher *and* sends its folder to the operating system Trash — recoverable there, not a hard, permanent delete. The very first tracker (the one **_app/** is physically nested inside — see Section 2) is the one exception: it can't be deleted this way, since removing it would mean removing the application's own home. Rename it instead if you want to repurpose it.

Keep the distinction clear: a workspace is a logical tracker inside the running application; a separate installation is a separate copy of the application code and its local databases, made by copying **_app/** into another tracker root.

8 Privacy and Local Data

JobTracker Hub is designed around local execution.

8.1 Network surface

The API is configured to bind to `127.0.0.1/localhost`. The web frontend is served by that same local FastAPI process and should be opened through `http://127.0.0.1:8000`, not as a bare local HTML file.

This architecture means the repository itself does not require a hosted backend, login, or cloud account.

8.2 Database management and local storage

overrides.db is the durable tracking database. It stores user-owned state such as manual status changes, notes, applied dates, next actions, snoozes, archives, company merges, and Search Hub settings. In contrast, **jobtracker.db** is the disposable index that is rebuilt from the filesystem. Rebuilding the index does not rewrite **overrides.db**.

The current repository documentation places each tracker's durable overrides database inside that tracker's own **.jobtracker/** directory. In other words, for a tracker rooted at **MyTracker/**, the standard tracker-local path is:

MyTracker/.jobtracker/overrides.db

This tracker-local arrangement is intentionally portable across operating systems: the database travels with the tracker rather than being tied to a particular user's home directory. The packaged macOS application also has per-user application state outside its read-only **.app** bundle; use **Settings > Diagnostics** to see the active state directory when troubleshooting or making a complete application-state backup.

Do not assume that copying only **overrides.db** is a complete backup. It contains the tracking metadata, while the documents it describes remain in the tracker filesystem.

8.2.1 Backup recommendations

- Back up `overrides.db` together with the tracker documents it describes. The database alone is not a substitute for the underlying filesystem.
- Prefer a copy made while JobTracker is not actively writing to the database, or otherwise use the application’s supported export workflow.
- Keep at least one backup outside the active tracker folder so accidental deletion or filesystem corruption does not remove both copies.
- Before a major migration or reinstall, export the tracker and retain the resulting archive in addition to your normal document backup.

8.3 What is stored locally

File	Purpose	Lifecycle
<code>jobtracker.db</code>	Indexed filesystem/application data	Rebuilt from disk whenever you rebuild the index.
<code>overrides.db</code>	Your manual tracking state	Durable; rebuilds deliberately preserve it.
<code>workspaces.json</code> / workspace state	Multi-tracker configuration	Local application state.

These files are gitignored in the repository because they can contain personal job-search data.

9 A Recommended Daily Workflow

The software becomes most useful when the filesystem and the tracking metadata have complementary jobs.

1. **Keep documents organized naturally.** Put the resume, cover letter, job posting, interview notes, and confirmation material for an application under the same company/role area.
2. **Rebuild after structural changes.** If you add or move a large batch of files in Finder/Explorer, rebuild the index so the dashboard reflects the filesystem.
3. **Record the meaningful dates.** Set Date applied rather than relying on file modification dates.
4. **Use Needs Attention as the morning queue.** Handle due follow-ups and stale applications first.
5. **Keep source documents on disk.** Let JobTracker add structure around them instead of replacing your existing document-management habits.
6. **Archive instead of deleting.** Use archive for old applications you want out of the active workflow while retaining their history.

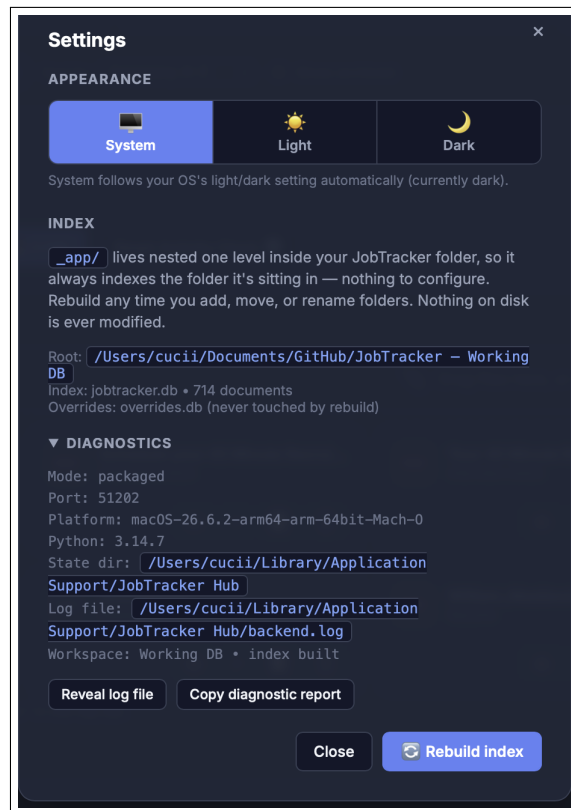
10 Troubleshooting

10.1 The desktop app won’t open, or quits immediately

The desktop build keeps a log of its startup backend and, if it can’t start, shows a dialog explaining why instead of quitting silently.

1. If a dialog appears, read it — it names the problem and points to the log file directly.
2. If nothing appears at all, open **JobTracker Hub**, click the **J** logo, choose **Settings**, and expand **Diagnostics**. This shows the port, state directory, and log file path for the current session, plus a **Reveal log file** button and a **Copy diagnostic report** button for sharing what’s wrong.

3. Relaunch the app once. A single failed launch is occasionally a slow-machine timing issue; a repeated failure is worth checking the log for.



Settings > Diagnostics: index root, state directory, log file path, and the report/rebuild actions.

10.2 The dashboard is empty

Confirm that:

1. `_app/` is nested exactly one level inside the tracker root.
2. The tracker root contains at least one real, non-ignored document.
3. You clicked **Build index** after starting the app.

10.3 A folder appears in the wrong section

Edit `_app/classify_config.json`, adjust the section rule, and rebuild. Do not rewrite `classify.py` unless you are changing filename-level document-type heuristics.

10.4 My notes or status changed unexpectedly after a rebuild

A rebuild is not supposed to erase `overrides.db`. Check that the file still exists in the active tracker and that you have not created or switched to a different workspace.

10.5 The browser cannot reach the API

Verify that the terminal running Uvicorn is still active and that it is listening on port 8000. A minimal command is:

```
uvicorn api:app --reload --port 8000
```

Then refresh the frontend.

10.6 A document is missing

First check whether it is hidden because it is a source file such as `.tex`. Then rebuild the index. Also check whether the filename, folder, or extension matches one of the ignored patterns documented in `classify.py`.

11 License

JobTracker Hub is distributed under the MIT License. See the repository's `LICENSE` file for the authoritative license text.

A Repository Map for Contributors

This appendix is intentionally outside the primary user journey. It is useful when maintaining, packaging, or extending JobTracker Hub, but most users never need it. The current repository keeps the implementation intentionally small and readable:

File / directory	Role
<code>_app/api.py</code>	FastAPI backend and local file/API operations.
<code>_app/db.py</code>	Data access, effective status/company state, staleness, and Insights calculations.
<code>_app/build_index.py</code>	Filesystem parser and disposable index builder.
<code>_app/overrides_store.py</code>	Durable user-owned tracking overrides.
<code>_app/classify.py</code>	Filename and section heuristics.
<code>_app/classify_config.json</code>	User-editable folder-to-section rules.
<code>_app/frontend/index.html</code>	Single-file React-based web frontend, loaded from a CDN.
<code>_app/workspace.py</code>	Multiple-tracker/workspace bookkeeping.
<code>_app/labels.py</code>	Section and document-type display labels used by the frontend.
<code>desktop/launcher.py</code>	pywebview bridge for native folder pickers and preview/confirm operations.
<code>desktop/first_run.html</code>	First-run window for a fresh packaged desktop install.
<code>scripts/fix_doubled_tracker_names.py</code>	One-off cleanup for older doubled tracker names; dry run by default.
<code>sample-tracker/</code>	Safe fake data for first-run exploration.

B Repository Safety Notes

JobTracker can manipulate real files, so a few conventions are intentionally strict:

- Path resolution in file endpoints is constrained to the tracker root.
- The indexer skips its own application directory and common generated artifacts.
- Database files are gitignored and should not be committed as part of normal development.
- The sample tracker uses fake organizations and placeholder content so you can test without exposing real applications.

C Release Notes for 1.1.0

This guide is aligned to the repository README supplied with JobTracker Hub 1.1.0. The release documentation covers the current macOS DMG installation and Gatekeeper flow, the tracker switcher and preview-first import/link workflow, durable local tracking data in `overrides.db`, Search Hub persistence, and the current packaging/test commands.

The guide also documents the workspace status cards and import validation sequence in more detail so that the user-facing manual explains the same current workflows from the perspective of someone operating the application.

For future releases, update this section and the guide version metadata together with the repository README and the shipped PDF.